IPSSI

2e année Bachelor Informatique

Mathématiques appliquées à l'Intelligence Artificielle

Corrections détaillées

Jour 4 — Réseaux de neurones

Auteur : Mohamed EL AFRIT

www.mohamedelafrit.com

Année académique : 2025–2026

© 2025 Mohamed EL AFRIT — IPSSI

Ce contenu est distribué sous licence

Creative Commons BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.fr

Rappels

$\sigma(z) = \frac{1}{1+e^{-z}}$ $\quad \mathrm{ReLU}(z) = \max(0, z)$ $\quad \sigma'(z) = \sigma(z)(1-\sigma(z))$ $\quad \sigma(0) = 0{,}5$ $\quad \sigma(1) \approx 0{,}73$
$\sigma(2) \approx 0{,}88$ $\quad \sigma(-1) \approx 0{,}27$

Exercice 1

★☆☆ **Fondamental**

$\mathbf{w} = (0{,}3;\ 0{,}5)^T$, $b = -2$, *activation sigmoïde.*

a) Somme pondérée pour $\mathbf{x} = (5, 4)^T$:

$$z = 0{,}3 \times 5 + 0{,}5 \times 4 + (-2) = 1{,}5 + 2{,}0 - 2{,}0 = \boxed{1{,}5}$$

b) Activation : $a = \sigma(1{,}5) = \frac{1}{1+e^{-1{,}5}} = \frac{1}{1+0{,}2231} = \frac{1}{1{,}2231} = \boxed{0{,}8176}$

c) $a = 0{,}818 > 0{,}5$ donc le neurone prédit la **classe 1**.

d) Pour $\mathbf{x} = (1, 2)^T$: $z = 0{,}3 \times 1 + 0{,}5 \times 2 - 2 = 0{,}3 + 1{,}0 - 2{,}0 = -0{,}7$.
$a = \sigma(-0{,}7) = \frac{1}{1+e^{0{,}7}} = \frac{1}{1+2{,}014} = \boxed{0{,}332}$. Comme $0{,}332 < 0{,}5$, le neurone prédit la **classe 0**.

⚠ Erreurs fréquentes

- Oublier le biais $b = -2$ dans le calcul de z.
- Confondre e^{-z} et e^{z} dans la formule de la sigmoïde : c'est e^{-z} au dénominateur.

Exercice 2

★☆☆ **Fondamental**

a) Sigmoïde :
$\sigma(0) = \frac{1}{1+e^{0}} = \frac{1}{2} = \boxed{0{,}5}$
$\sigma(2) = \frac{1}{1+e^{-2}} = \frac{1}{1+0{,}135} = \boxed{0{,}881}$
$\sigma(-3) = \frac{1}{1+e^{3}} = \frac{1}{1+20{,}09} = \boxed{0{,}047}$
b) ReLU :
$\mathrm{ReLU}(2{,}5) = \max(0,\ 2{,}5) = \boxed{2{,}5}$
$\mathrm{ReLU}(0) = \max(0,\ 0) = \boxed{0}$
$\mathrm{ReLU}(-1{,}7) = \max(0,\ -1{,}7) = \boxed{0}$

c) $\mathrm{ReLU}(-1) = \max(0, -1) = 0$, puis $\sigma(0) = 0{,}5$. Donc $\sigma(\mathrm{ReLU}(-1)) = \boxed{0{,}5}$.

d) $\mathrm{ReLU}(3) = 3$, puis $\sigma(3) = \frac{1}{1+e^{-3}} \approx 0{,}953$. Donc $\sigma(\mathrm{ReLU}(3)) = \boxed{0{,}953}$.

e) $\sigma(z) = \frac{1}{1+e^{-z}}$. Le dénominateur $1 + e^{-z}$ est toujours > 1 (car $e^{-z} > 0$ pour tout z), donc $\sigma(z) < 1$. De même, $\sigma(z) > 0$ car le numérateur et le dénominateur sont positifs. La sigmoïde est **asymptotique** : elle s'approche de 0 et 1 sans jamais les atteindre.

 CC BY-NC-SA 4.0

À retenir

ReLU "coupe" les valeurs négatives à zéro. La sigmoïde "écrase" tout nombre réel dans $(0, 1)$. Composer les deux crée une fonction non linéaire.

Exercice 3

★★☆ **Fondamental**

$\mathbf{x} = (3, 5)^T$, $\mathbf{W}^{(1)}$, $\mathbf{w}^{(2)}$ *donnés.*

a) Sommes pondérées couche cachée :

$$z_1 = 0{,}4 \times 3 + 0{,}6 \times 5 + (-1) = 1{,}2 + 3{,}0 - 1 = \boxed{3{,}2}$$
$$z_2 = (-0{,}3) \times 3 + 0{,}5 \times 5 + 0{,}5 = -0{,}9 + 2{,}5 + 0{,}5 = \boxed{2{,}1}$$

b) Activations : $h_1 = \sigma(3{,}2) \approx \boxed{0{,}961}$; $h_2 = \sigma(2{,}1) \approx \boxed{0{,}891}$

c) Somme pondérée sortie :

$$z_{\text{out}} = 0{,}7 \times 0{,}961 + 0{,}8 \times 0{,}891 + (-0{,}5) = 0{,}673 + 0{,}713 - 0{,}5 = \boxed{0{,}886}$$

d) Prédiction : $\hat{y} = \sigma(0{,}886) \approx \boxed{0{,}708}$

e) $0{,}708 > 0{,}5$ donc le réseau prédit la **classe 1** (salaire $> 45\,\text{k€}$).

Erreurs fréquentes

- Utiliser $\mathbf{x}$ directement dans la couche de sortie au lieu de $\mathbf{h}$ — la sortie utilise les **activations de la couche cachée**, pas les entrées.
- Oublier d'appliquer σ à chaque couche.

Exercice 4

★★☆ **Fondamental**

Réseau 5 → 8 → 3 → 1.

a) $\mathbf{W}^{(1)} \in \mathbb{R}^{8\times 5}$ (8 neurones, 5 entrées). $\mathbf{b}^{(1)} \in \mathbb{R}^8$.

b) $\mathbf{W}^{(2)} \in \mathbb{R}^{3\times 8}$ (3 neurones, 8 entrées). $\mathbf{b}^{(2)} \in \mathbb{R}^3$.

c) $\mathbf{w}^{(3)} \in \mathbb{R}^3$ (ou $\mathbf{W}^{(3)} \in \mathbb{R}^{1\times 3}$). $b^{(3)} \in \mathbb{R}$.

d) Total des paramètres :

$$\underbrace{8 \times 5}_{W^{(1)}} + \underbrace{8}_{b^{(1)}} + \underbrace{3 \times 8}_{W^{(2)}} + \underbrace{3}_{b^{(2)}} + \underbrace{1 \times 3}_{w^{(3)}} + \underbrace{1}_{b^{(3)}}$$
$$= 40 + 8 + 24 + 3 + 3 + 1 = \boxed{79 \text{ paramètres}}$$

e) Couche 1 passe de 8 à 16 neurones : $W^{(1)}$ passe de $8 \times 5 = 40$ à $16 \times 5 = 80$ $(+40)$. $b^{(1)}$ passe de 8 à 16 $(+8)$. $W^{(2)}$ passe de $3 \times 8 = 24$ à $3 \times 16 = 48$ $(+24)$. Total supplémentaire : $\boxed{+72}$ paramètres (de 79 à 151).

À retenir

Le nombre de paramètres croît **quadratiquement** avec la taille des couches. C'est pourquoi les grands réseaux (GPT, etc.) ont des milliards de paramètres.

Exercice 5

★★☆ Intermédiaire

$f_A(x) = ReLU(2x-1)$, $g_B(x) = \sigma(3x-2)$.

---

a) $f_A(0) = \text{ReLU}(-1) = \boxed{0}$; $f_A(1) = \text{ReLU}(1) = \boxed{1}$; $f_A(3) = \text{ReLU}(5) = \boxed{5}$.

b) Composition $g_B(f_A(x))$ **:**

$x = 0$: $f_A(0) = 0$, puis $g_B(0) = \sigma(3 \times 0 - 2) = \sigma(-2) \approx \boxed{0{,}119}$.

$x = 1$: $f_A(1) = 1$, puis $g_B(1) = \sigma(3 \times 1 - 2) = \sigma(1) \approx \boxed{0{,}731}$.

$x = 3$: $f_A(3) = 5$, puis $g_B(5) = \sigma(3 \times 5 - 2) = \sigma(13) \approx \boxed{1{,}000}$.

c) Non, $g_B \circ f_A$ n'est **pas** linéaire. Elle combine ReLU (linéaire par morceaux) et sigmoïde (courbe en S). La composition de fonctions non linéaires est non linéaire.

d) Sans activation : $f(x) = 2x - 1$, $g(x) = 3x - 2$. $g(f(x)) = 3(2x-1) - 2 = 6x - 3 - 2 = \boxed{6x - 5}$. C'est une fonction **linéaire** — deux couches sans activation = une seule couche.

À retenir

C'est la preuve concrète : sans activation, empiler des couches ne sert à rien.

Exercice 6

★★☆ Intermédiaire

---

a-b) Substitution de $\mathbf{h}$ dans $\hat{y}$:

$$\begin{aligned}\hat{y} &= \mathbf{w}^{(2)T}\mathbf{h} + b^{(2)} \\ &= \mathbf{w}^{(2)T}\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right) + b^{(2)} \\ &= \underbrace{\mathbf{w}^{(2)T}\mathbf{W}^{(1)}}_{\tilde{\mathbf{w}}^T}\mathbf{x} + \underbrace{\mathbf{w}^{(2)T}\mathbf{b}^{(1)} + b^{(2)}}_{\tilde{b}}\end{aligned}$$

Donc $\hat{y} = \tilde{\mathbf{w}}^T\mathbf{x} + \tilde{b}$ avec $\tilde{\mathbf{w}} = \mathbf{W}^{(1)T}\mathbf{w}^{(2)}$.

c) C'est un **modèle linéaire** — exactement comme un perceptron simple. Le réseau à 2 couches sans activation n'est **pas plus puissant** qu'un seul neurone.

d) Les fonctions d'activation sont indispensables car elles introduisent la **non-linéarité** qui permet de modéliser des frontières courbes et de résoudre des problèmes non linéairement séparables (comme le XOR).

Exercice 7

★★☆ Intermédiaire

 CC BY-NC-SA 4.0

a) Le surapprentissage commence vers l'**époque 200**. Signe : l'accuracy d'entraînement continue d'augmenter (95% → 100%) tandis que l'accuracy de validation **diminue** (87% → 75%). L'écart train/validation se creuse.

b) Le meilleur modèle est à l'**époque 50** : accuracy de validation maximale (83%), avec un écart raisonnable par rapport à l'entraînement (85% vs 83%).

c) Trois solutions :

1. **Early stopping** : arrêter l'entraînement quand la validation cesse de s'améliorer.
2. **Régularisation** L_2 : ajouter un terme $\lambda\|\mathbf{w}\|^2$ à la loss pour pénaliser les poids trop grands.
3. **Dropout** : désactiver aléatoirement des neurones pendant l'entraînement pour forcer la redondance.

d) 100% sur l'entraînement signifie que le modèle a **mémorisé** toutes les observations, y compris le bruit et les cas atypiques. C'est comme un élève qui apprend ses fiches par cœur sans comprendre : il échoue face à un problème nouveau.

⚠ Erreurs fréquentes

— Choisir le modèle avec la meilleure accuracy d'**entraînement** — il faut regarder la **validation**.

— Croire qu'un modèle plus complexe est toujours meilleur : au-delà d'un certain point, il surapprentit.

Exercice 8

★★☆ Intermédiaire

» Correction complète

```python
import numpy as np
import matplotlib.pyplot as plt

# 1. Sigmoide
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# 2. ReLU
def relu(z):
    return np.maximum(0, z)

# 3. Test
valeurs = np.array([-3, -1, 0, 1, 2, 5])
print("z       :", valeurs)
print("sigmoid :", np.round(sigmoid(valeurs), 4))
print("relu    :", relu(valeurs))

# 4. Courbes
```

```python
z = np.linspace(-6, 6, 200)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(z, sigmoid(z), 'b-', lw=2)
ax1.set_title("Sigmoide"); ax1.grid(True, alpha=0.3)
ax1.axhline(0.5, color='gray', ls=':')

ax2.plot(z, relu(z), 'r-', lw=2)
ax2.set_title("ReLU"); ax2.grid(True, alpha=0.3)

plt.tight_layout(); plt.show()

# 5. Derivee
def sigmoid_deriv(z):
    s = sigmoid(z)
    return s * (1 - s)

print("\nDerivee sigmoid(0) =", sigmoid_deriv(0))
print("Derivee sigmoid(2) =", round(sigmoid_deriv(2), 4))
```

Sortie attendue :

```
z       : [-3 -1  0  1  2  5]
sigmoid : [0.0474 0.2689 0.5    0.7311 0.8808 0.9933]
relu    : [0 0 0 1 2 5]

Derivee sigmoid(0) = 0.25
Derivee sigmoid(2) = 0.1050
```

Variante

Essayez aussi `np.tanh(z)` (tangente hyperbolique, sortie dans $(-1, 1)$) et la **Leaky ReLU** : `np.where(z > 0, z, 0.01 * z)`.

Exercice 9

★★☆ Intermédiaire

» Correction complète

```python
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

x = np.array([5.0, 4.0])

W1 = np.array([[0.3, 0.5], [-0.2, 0.4]])
```

```python
b1 = np.array([-2.0, 0.5])
w2 = np.array([0.8, 0.6])
b2 = -0.5

# 1. Couche cachee
z1 = W1 @ x + b1
h  = sigmoid(z1)
print(f"z1 = {z1}")
print(f"h  = {np.round(h, 4)}")

# 2. Couche de sortie
z2 = w2 @ h + b2
y_pred = sigmoid(z2)
print(f"\nz2 = {z2:.4f}")
print(f"y_pred = {y_pred:.4f}")

# 3. Decision
classe = 1 if y_pred > 0.5 else 0
print(f"\nClasse predite : {classe}")

# 4. Test sur 6 developpeurs
print("\n--- Predictions ---")
devs = np.array([[0,2],[1,1],[5,4],[9,5],[12,7],[18,5]])
for d in devs:
    h_i = sigmoid(W1 @ d + b1)
    y_i = sigmoid(w2 @ h_i + b2)
    print(f"  x={d} -> y={y_i:.3f} -> "
          f"classe {1 if y_i > 0.5 else 0}")
```

Sortie attendue :

```
z1 = [1.5 1.1]
h  = [0.8176 0.7503]

z2 = 0.6043
y_pred = 0.6466

Classe predite : 1

--- Predictions ---
  x=[0 2] -> y=0.551 -> classe 1
  x=[1 1] -> y=0.533 -> classe 1
  x=[5 4] -> y=0.647 -> classe 1
  x=[9 5] -> y=0.693 -> classe 1
  x=[12 7] -> y=0.726 -> classe 1
  x=[18 5] -> y=0.715 -> classe 1
```

⚠ Erreurs fréquentes

- Écrire `w2 @ x + b2` au lieu de `w2 @ h + b2` — la sortie utilise `h`, pas `x`.
- Avec ces poids non entraînés, le réseau prédit "classe 1" pour tout le monde — c'est normal, il n'a pas appris.

Exercice 10

★★☆ Intermédiaire

» Correction complète

```python
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))
def sigmoid_d(a):
    return a * (1 - a)

X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([[0],[1],[1],[0]], dtype=float)

np.random.seed(42)
W1 = np.random.randn(2, 4) * 0.5
b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1) * 0.5
b2 = np.zeros((1, 1))

alpha = 1.0

for epoch in range(5000):
    # Forward
    z1 = X @ W1 + b1
    h  = sigmoid(z1)
    z2 = h @ W2 + b2
    out = sigmoid(z2)

    loss = np.mean((y - out) ** 2)

    # Backward
    d_out = (out - y) * sigmoid_d(out)
    dW2 = h.T @ d_out / 4
    db2 = np.mean(d_out, axis=0, keepdims=True)

    d_h = d_out @ W2.T * sigmoid_d(h)
    dW1 = X.T @ d_h / 4
    db1 = np.mean(d_h, axis=0, keepdims=True)

    # Mise a jour
    W2 -= alpha * dW2; b2 -= alpha * db2
```

```
    W1 -= alpha * dW1; b1 -= alpha * db1

    if epoch % 1000 == 999:
        preds = (out > 0.5).astype(int).flatten()
        acc = np.mean(preds == y.flatten())
        print(f"Epoch {epoch+1} : "
              f"loss={loss:.4f}, acc={acc:.0%}")

print(f"\nPredictions : {out.flatten().round(3)}")
print(f"Attendu     : {y.flatten().astype(int)}")
```

Sortie attendue :

```
Epoch 1000 : loss=0.2336, acc=75%
Epoch 2000 : loss=0.0124, acc=100%
Epoch 3000 : loss=0.0048, acc=100%
Epoch 4000 : loss=0.0027, acc=100%
Epoch 5000 : loss=0.0018, acc=100%

Predictions : [0.039 0.965 0.965 0.038]
Attendu     : [0 1 1 0]
```

À retenir

Le réseau 2-4-1 résout le XOR avec 100% d'accuracy ! Les 4 neurones cachés créent des **représentations intermédiaires** qui rendent le problème linéairement séparable dans $\mathbb{R}^4$.

Exercice 11

★★★ Avancé

» Correction complète

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))
def sigmoid_d(a):
    return a * (1 - a)

# a) Dataset
data = np.array([
    [0,2,28],[1,1,26.5],[1,3,35],[2,2,32],[2,4,38],
    [0,1,25],[3,3,34.5],[1,5,42],[4,3,38.5],[5,4,44],
    [5,3,41],[6,5,48],[6,4,43.5],[7,5,50],[7,3,39],
    [4,6,47],[8,4,51],[5,2,40],[9,5,52],[10,6,55],
```

```
    [10,4,48.5],[11,5,58],[12,7,60],[9,3,42],
    [13,6,62],[11,4,54],[15,8,65],[17,6,68],
    [20,7,72],[18,5,58]], dtype=float)

X_raw = data[:, :2]
y = (data[:, 2] > 45).astype(float).reshape(-1, 1)

# b) Normalisation
mu = X_raw.mean(axis=0)
sigma_x = X_raw.std(axis=0)
X = (X_raw - mu) / sigma_x

# c) Reseau 2-4-1
np.random.seed(42)
W1 = np.random.randn(2, 4) * 0.5
b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1) * 0.5
b2 = np.zeros((1, 1))
alpha = 0.5
n = len(y)
hist_loss = []

for epoch in range(2000):
    h = sigmoid(X @ W1 + b1)
    out = sigmoid(h @ W2 + b2)
    loss = np.mean((y - out) ** 2)
    hist_loss.append(loss)

    d_out = (out - y) * sigmoid_d(out)
    W2 -= alpha * (h.T @ d_out) / n
    b2 -= alpha * np.mean(d_out, axis=0, keepdims=True)
    d_h = (d_out @ W2.T) * sigmoid_d(h)
    W1 -= alpha * (X.T @ d_h) / n
    b1 -= alpha * np.mean(d_h, axis=0, keepdims=True)

    if epoch % 500 == 499:
        acc = np.mean((out > 0.5) == y)
        print(f"Epoch {epoch+1:5d} : "
              f"loss={loss:.4f}, acc={acc:.0%}")

# e) Graphiques
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
ax1.plot(hist_loss, 'b-', lw=1)
ax1.set_xlabel("Epoch"); ax1.set_ylabel("Loss")
ax1.set_title("Convergence"); ax1.grid(True, alpha=0.3)

# Surface de decision
xx, yy = np.meshgrid(
    np.linspace(X[:,0].min()-1, X[:,0].max()+1, 100),
    np.linspace(X[:,1].min()-1, X[:,1].max()+1, 100))
grid = np.c_[xx.ravel(), yy.ravel()]
h_g = sigmoid(grid @ W1 + b1)
z_g = sigmoid(h_g @ W2 + b2).reshape(xx.shape)
```

 CC BY-NC-SA 4.0

```
ax2.contourf(xx, yy, z_g, levels=20,
             cmap='RdYlGn', alpha=0.5)
ax2.contour(xx, yy, z_g, levels=[0.5],
            colors='black', linewidths=2)
for c, col in [(0,'#e74c3c'),(1,'#2ecc71')]:
    mask = y.flatten() == c
    ax2.scatter(X[mask,0], X[mask,1], c=col,
                s=60, edgecolors='white', lw=0.5)
ax2.set_title("Surface de decision")
ax2.set_xlabel("Exp (norm)")
ax2.set_ylabel("Lang (norm)")
plt.tight_layout(); plt.show()
```

Sortie attendue :

```
Epoch   500 : loss=0.1073, acc=90%
Epoch  1000 : loss=0.0681, acc=93%
Epoch  1500 : loss=0.0523, acc=93%
Epoch  2000 : loss=0.0435, acc=97%
```

L'accuracy (93–97%) dépasse celle du perceptron (∼87–90%). La frontière est **courbe**, ce qui permet de mieux séparer les cas atypiques.

⚠ Erreurs fréquentes

- Oublier de **normaliser** les features — sans normalisation, le réseau converge très lentement ou pas du tout.
- Utiliser `np.clip` dans la sigmoïde pour éviter les overflow numériques sur de grandes valeurs.

Exercice 12

★★★ Avancé

» Correction complète

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import Perceptron as SkPerc
from sklearn.preprocessing import StandardScaler

data = np.array([
    [0,2,28],[1,1,26.5],[1,3,35],[2,2,32],[2,4,38],
    [0,1,25],[3,3,34.5],[1,5,42],[4,3,38.5],[5,4,44],
    [5,3,41],[6,5,48],[6,4,43.5],[7,5,50],[7,3,39],
    [4,6,47],[8,4,51],[5,2,40],[9,5,52],[10,6,55],
    [10,4,48.5],[11,5,58],[12,7,60],[9,3,42],
```


CC BY-NC-SA 4.0

```
    [13,6,62],[11,4,54],[15,8,65],[17,6,68],
    [20,7,72],[18,5,58]], dtype=float)

X_raw = data[:,:2]
y = (data[:,2] > 45).astype(int)
scaler = StandardScaler()
X = scaler.fit_transform(X_raw)

# M1: From scratch (meme code que ex 11)
def sig(z): return 1/(1+np.exp(-np.clip(z,-500,500)))
def sig_d(a): return a*(1-a)
np.random.seed(42)
W1=np.random.randn(2,4)*0.5; b1=np.zeros((1,4))
W2=np.random.randn(4,1)*0.5; b2=np.zeros((1,1))
yr = y.reshape(-1,1).astype(float)
for _ in range(2000):
    h=sig(X@W1+b1); out=sig(h@W2+b2)
    do=(out-yr)*sig_d(out)
    W2-=0.5*(h.T@do)/30; b2-=0.5*np.mean(do,axis=0,keepdims=True)
    dh=(do@W2.T)*sig_d(h)
    W1-=0.5*(X.T@dh)/30; b1-=0.5*np.mean(dh,axis=0,keepdims=True)
acc_fs = np.mean((out>0.5).flatten()==y)

# M2: MLPClassifier
mlp = MLPClassifier(hidden_layer_sizes=(4,),
    max_iter=2000, activation='logistic',
    random_state=42)
mlp.fit(X, y); acc_mlp = mlp.score(X, y)

# M3: Perceptron
perc = SkPerc(max_iter=100, eta0=0.1,
              random_state=42)
perc.fit(X, y); acc_perc = perc.score(X, y)

# c) Tableau
print(f"{'Methode':<22}|{'Acc':>6}|{'Params':>7}")
print("-" * 38)
print(f"{'From scratch 2-4-1':<22}|{acc_fs:>5.0%}|"
      f"{2*4+4+4*1+1:>7}")
print(f"{'MLPClassifier (4,)':<22}|{acc_mlp:>5.0%}|"
      f"{2*4+4+4*1+1:>7}")
print(f"{'Perceptron':<22}|{acc_perc:>5.0%}|"
      f"{'3':>7}")

# d) 3 surfaces de decision
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
xx, yy = np.meshgrid(
    np.linspace(X[:,0].min()-1, X[:,0].max()+1, 100),
    np.linspace(X[:,1].min()-1, X[:,1].max()+1, 100))
grid = np.c_[xx.ravel(), yy.ravel()]

# From scratch
h_g=sig(grid@W1+b1)
```

 CC BY-NC-SA 4.0

```python
z_fs=sig(h_g@W2+b2).reshape(xx.shape)
# MLPClassifier
z_mlp=mlp.predict_proba(grid)[:,1].reshape(xx.shape)
# Perceptron
z_perc=perc.decision_function(grid).reshape(xx.shape)

titles = [f'From scratch ({acc_fs:.0%})',
          f'MLPClassifier ({acc_mlp:.0%})',
          f'Perceptron ({acc_perc:.0%})']
zz = [z_fs, z_mlp, z_perc]
lvls = [[0.5], [0.5], [0]]

for ax, z, t, lv in zip(axes, zz, titles, lvls):
    ax.contourf(xx, yy, z, levels=20,
                cmap='RdYlGn', alpha=0.4)
    ax.contour(xx, yy, z, levels=lv,
               colors='black', linewidths=2)
    for c, col in [(0,'#e74c3c'),(1,'#2ecc71')]:
        mask = y == c
        ax.scatter(X[mask,0], X[mask,1], c=col,
                   s=50, edgecolors='white', lw=0.5)
    ax.set_title(t); ax.grid(True, alpha=0.2)
plt.tight_layout(); plt.show()

# e) Le reseau est meilleur que le perceptron.
# Sa frontiere est COURBE (non lineaire).
# Le perceptron a 3 params, le reseau 17.
```

Sortie attendue :

```
Methode                |   Acc| Params
--------------------------------------
From scratch 2-4-1     |  97% |     17
MLPClassifier (4,)     |  93% |     17
Perceptron             |  87% |      3
```

Conclusion générale du Jour 4

Le réseau multicouche est strictement plus puissant que le perceptron : sa frontière **courbe** capture les relations non linéaires. Le prix à payer : plus de paramètres (17 vs 3), plus de temps d'entraînement, et un risque de surapprentissage. C'est le **compromis biais–variance** fondamental du ML.

Bilan des 4 jours

Vous avez construit, **from scratch**, toute la chaîne de l'IA : **J1** vecteurs et matrices, **J2** descente de gradient, **J3** perceptron, **J4** réseau de neurones. Chaque framework moderne (TensorFlow, PyTorch) implémente exactement ces concepts — en plus rapide et plus grand. Vous avez les fondations pour les comprendre.

© 2025 Mohamed EL AFRIT — IPSSI • www.mohamedelafrit.com • CC BY-NC-SA 4.0